Adria Mongar

Testing and Debugging

Actual Time: 1 Hour
Summary: *Making sure everything worked and was up to requirements*

Daniel Harris

Testing and Debugging

Description: *Implement functionality for storing records of all previous times and display them after a game finishes. Display the 5 best times and indicate if the current score is a new record.*
Estimated Time: 2 Hours
Actual Time: 2 Hours *The implementation was straight forward simply involving manipulation of a text file that store records and a function to retrieve them. No major problems were encountered while implementing the functionality.*

Eliza Malyshev

Testing and Debugging

Estimated Time: 3 Hours
Actual Time: 2.5 Hours
Summary: *Tested the game for general issues before feature adding and after feature adding. updated the readme to include instructions for linux setup for the game.*

Emilia Davis

Hard AI

Estimated Time: 2 Hours
Actual Time: 3 Hours *The actual coding of the module was harder than I expected and it took me a bit to fully understand everything and comment it.*
Summary: *Created hard AI module that implements medium ruleset, 1-2-1 rule, and falls back to random. UI dropdown/Combining AI solvers with main Est: 1hr Actual 1hr Had some minor difficulty with the style of the dropdown but nothing beyond what I expected. Summary: connected main file with ai_solvers file.*

Jett Viduya

Easy-Medium AI logic, testing AI logic integration (one testing script).

Estimated Time: 3-4 Hours
Actual time: 3-4 Hours
Summary: *Implemented the Easy and Medium AI modes in ai_solver.py, building Medium on top of Easy. Wrote test_ai_solver.py to test the AI's integration with the board and merged the completed work into main. Did further testing once other modes and UI elements were integrated.*

Ryan Graham

Minebrute Algorithm Implementation

Description: *The minebrute algorithm checks all possible mine configurations around each tile and identifies which configurations contradict revealed tiles' information. From this, we can narrow down the range of tiles which are absolutely safe or absolutely mines.*
Estimated Time: 3 Hours. *Writing an algorithm to do a brute force search over the board to identify tiles that are either absolutely safe or absolutely mines has complicated logic, and requires hundreds of lines of code to implement.*
Actual Time: 2 Hours.

Minebrute Algorithm - Integration, Debugging, Testing

Description: *This involved allowing the Minebrute algorithm to run on the board in actual play, and ensuring no issues arose from its execution.*
Estimated Time: 2 Hours. *The algorithm should be complete, so plugging it into the GitHub repository should be simple, provided no major bugs exist.*
Actual Time: 4 Hours. *There were in fact, bugs. These related to simple errors in function inputs, break statements not going to the right places in execution, processing on games that already ended, etc.*

Zachary Mccauley

Score Keeping

Description: *Implement functionality for storing records of all previous times and display them after a game finishes. Display the 5 best times and indicate if the current score is a new record.*
Estimated Time: 2 Hours
Actual Time: 2 hours The implementation was straight forward simply involving manipulation of a text file that store records and a function to retrieve them. No major problems were encountered while implementing the functionality

AI UI

Description: *Add menus to allow the user to select between "normal" "interactive" and "automatic" and a menu for the difficulties "easy" "medium" and "hard". The players choices from this menu should affect how the game runs and how the AI solver is called.*
Estimated Time: 2 Hours
Actual Time: 3 Hours *The highly modular design of the game caused adding new features to be overly extensive. Implementing delays so the AI's actions are readable by the player caused parts of the game to need to be redesigned.*